

# pyglenn: A Python Toolkit for Thermochemical Properties Calculation from NASA Polynomials

Dr. Reginaldo G. Leão Jr.

July 2026

## Abstract

This paper presents pyglenn, an open-source Python package designed for the computation of thermochemical properties — heat capacity ( $C_p$ ), enthalpy ( $H^\circ$ ), and entropy ( $S^\circ$ ) — from NASA-7 polynomial coefficients. The package converts legacy FORTRAN `thermo.inp` files into a normalized SQLite database and provides both a programmatic API and a command-line interface. With approximately 2,030 chemical species and 3,772 temperature intervals, pyglenn serves as a lightweight, dependency-free tool for researchers and engineers in combustion science, chemical thermodynamics, and related fields.

## 1. Introduction

Accurate thermochemical data is fundamental to modeling and simulating reactive flows, combustion processes, and chemical equilibrium. The NASA polynomial format, originally developed for the NASA Glenn Research Center’s chemical equilibrium program, has become a de facto standard for representing temperature-dependent thermodynamic properties of chemical species over wide temperature ranges [1, 2].

The `thermo.inp` files distributed with legacy FORTRAN codes contain polynomial coefficients for thousands of species, but their fixed-column FORTRAN format makes them cumbersome to parse in modern computational workflows. **pyglenn** bridges this gap by:

- Parsing FORTRAN-formatted `thermo.inp` files into a structured, normalized SQLite3 database;
- Providing a clean Python API for querying species and computing  $C_p(T)$ ,  $H^\circ(T)$ , and  $S^\circ(T)$  at arbitrary temperatures;
- Including a command-line interface for quick lookups;
- Bundling a pre-built database so that users require no manual setup.

The package is distributed under the MIT license and is available via PyPI (`pip install pyglenn`) and conda-forge (`conda install conda-forge::pyglenn`).

## 2. Theoretical Background

### 2.1 NASA-7 Polynomial Format

The NASA polynomial representation expresses dimensionless thermodynamic quantities as functions of temperature  $T$  [K] using seven coefficients ( $a_1, a_2, \dots, a_7$ ) per temperature interval:

$$\frac{C_p^\circ}{R} = a_1 + a_2T + a_3T^2 + a_4T^3 + a_5T^4 \quad (1)$$

$$\frac{H^\circ}{RT} = a_1 + \frac{a_2}{2}T + \frac{a_3}{3}T^2 + \frac{a_4}{4}T^3 + \frac{a_5}{5}T^4 + \frac{a_6}{T} \quad (2)$$

$$\frac{S^\circ}{R} = a_1 \ln T + a_2T + \frac{a_3}{2}T^2 + \frac{a_4}{3}T^3 + \frac{a_5}{4}T^4 + a_7 \quad (3)$$

where  $R = 8.314462618$  J/(mol · K) is the universal gas constant (CODATA 2018 value). Each species typically has two sets of coefficients — one for a low temperature interval and one for a high temperature interval — with a common midpoint temperature  $T_{\text{mid}}$ . The enthalpy  $H^\circ(T)$  is defined relative to the enthalpy of formation at 298.15 K.

### 2.2 The `thermo.inp` FORTRAN Format

The `thermo.inp` file follows a rigid fixed-column structure defined in the NASA Glenn report [1]. Each species occupies five record types:

| Record | Content                                                                                    |
|--------|--------------------------------------------------------------------------------------------|
| 1      | Species name and comments (A16, A62)                                                       |
| 2      | Number of intervals, chemical formula, phase, molecular weight, $\Delta H_f^\circ(298.15)$ |
| 3      | Temperature range ( $T_{\text{min}}$ , $T_{\text{max}}$ , $T_{\text{mid}}$ ), exponents    |
| 4      | First five coefficients $a_1$ – $a_5$ (5D16.9)                                             |
| 5      | Last two coefficients $a_6$ , $a_7$ and integration constants $b_1$ , $b_2$ (2D16.9 each)  |

Records 3–5 repeat for each temperature interval. The fixed-column nature and FORTRAN-specific notation (e.g., D instead of E in scientific notation) pose parsing challenges that `pyglenn` addresses through dedicated regular expressions and rigorous column-position extraction.

## 3. Results

### 3.1 Architecture and Database Design

pyglenn's architecture follows a layered design with three core modules:

- **builder.py** — Parses `thermo.inp` and populates a normalized SQLite database with four tables: `species`, `temperature_intervals`, `coefficients`, and `file_metadata`.
- **database.py** — Provides low-level SQL querying, species lookup, and the numerical evaluation of Equations (1)–(3).
- **calculator.py** — High-level interface exposing methods such as `calculate_properties()`, `calculate_formation_enthalpy()`, and `calculate_enthalpy_change()` with proper error handling.

The bundled database contains **approximately 2,030 species** spanning gaseous (G), liquid (L), and solid (S) phases, with **3,772 temperature intervals** covering ranges from cryogenic temperatures to several thousand Kelvin.

### 3.2 Python API

The primary interface is the `ThermochemicalCalculator` class, which supports the context-manager protocol for automatic resource management:

```
from pyglenn import ThermochemicalCalculator

with ThermochemicalCalculator() as calc:
    species = calc.get_available_species("CH4")
    props = calc.calculate_properties(species[0]["id"], 1000.0)
    print(f"Cp = {props['cp']:.2f} J/(mol·K)")
    print(f"H° = {props['h_relative']:.1f} J/mol")
    print(f"S° = {props['s']:.3f} J/(mol·K)")
```

No explicit database path is required — the package ships with a pre-built `thermo.db`, making it immediately usable after installation. Users can also supply a custom database file if desired.

### 3.3 Command-Line Interface

pyglenn provides a CLI for rapid inspection of thermochemical data:

```
# Build or rebuild the database from thermo.inp
pyglenn build -i thermo.inp -o thermo.db

# Query a species
pyglenn query -s O2
```

The build subcommand is only needed when users modify the source `thermo.inp` file or wish to regenerate a corrupted database.

### 3.4 Validation

The calculator enforces strict validation: each temperature request is checked against the species' valid intervals, and out-of-range values raise a `TemperatureOutOfRangeError`. Species not found in the database raise `SpeciesNotFoundError`. These guardrails ensure that users receive reliable results and are alerted when operating outside the valid domain of the polynomial fit.

## 4. Conclusion

pyglenn fills a practical niche in the computational thermodynamics ecosystem: it transforms the widely used but format-locked NASA polynomial data into an accessible, scriptable, and dependency-free Python tool. By bundling a pre-built SQLite database, it eliminates the friction of manual data conversion and lets researchers focus on their scientific questions.

The package is lightweight (zero runtime dependencies beyond the Python standard library), easy to install via PyPI or conda-forge, and provides both an intuitive Python API and a command-line interface. Future work may include support for the NASA-9 polynomial format (which uses nine coefficients per interval), integration with chemical equilibrium solvers, and a web-based interactive species browser.

---

## References

- [1] B. J. McBride, M. J. Zehe, and S. Gordon, "NASA Glenn Coefficients for Calculating Thermodynamic Properties of Individual Species," NASA/TP—2002-211556, NASA Glenn Research Center, Cleveland, OH, September 2002.
- [2] S. Gordon and B. J. McBride, "Computer Program for Calculation of Complex Chemical Equilibrium Compositions and Applications," NASA RP-1311, NASA Lewis Research Center, Cleveland, OH, October 1994.